Les algorithmes de reconnaissance de constellation

Thème : cycles et boucles



Chausson--Vergne Adam
Numéro de candidat : 33855

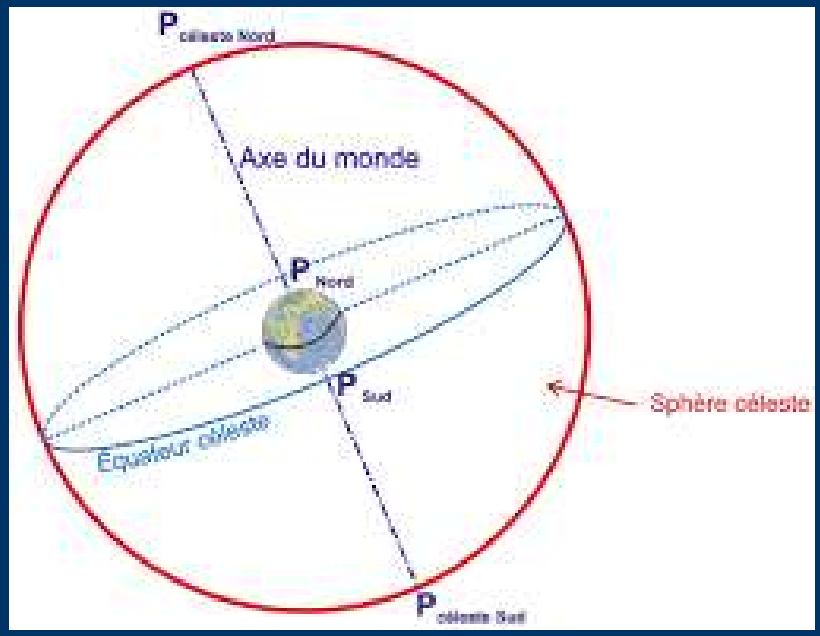
Veille technologie : les viseurs d'étoiles

Viseur d'étoile développé par la Sodern



https://sodern.com/fr/categories/viseurs-d-etoiles

Capteur d'orientation présent dans :
- Satellites de télécommunication
- Satellites GPS
- Sondes spatiales



Problématique et objectifs

Comment reconnaître des constellations d'étoiles sur des images ?

1) Conception de l'algorithme

```
def affiche2(Photo,coord): # sert à placer les points des consts générées
    m, n = Photo.shape[:2]
    for i in range(m):
        for j in range(n):
            Photo[i,j,0], Photo[i,j,1], Photo[i,j,2] = 0, 0, 0
    # for k in [0,1]:
    for i in range(len(coord)): # len(coord[k])
        for j1 in range(-3,3):
            for j2 in range(-3,3):
                Photo[coord[i][0]+j1, coord[i][1]+j2, 0] = 1 # ajouter [k] entre coord et [i][0]
                Photo[coord[i][0]+j1, coord[i][1]+j2, 1] = 1
                Photo[coord[i][0]+j1, coord[i][1]+j2, 2] = 1
```

2) Performances simulées



3) Performances réelles

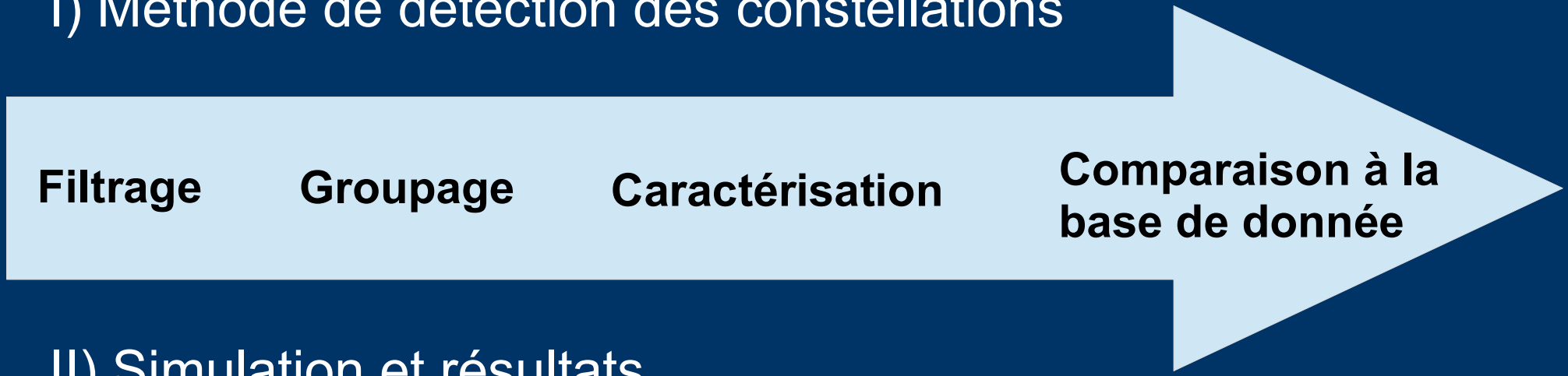


Cahier des charges du programme

Cahier des charges partiel	
Critère	Niveau
Insensibilité aux perturbations	Tolérance > 1.0% de distorsion de l'image
Taux de détection (T)	T > 90%
Temps d'exécution du code (Tr)	Tr < 1.0s

Sommaire

I) Méthode de détection des constellations



II) Simulation et résultats
- 1) Insensibilité aux perturbations
- 2) Taux de détection

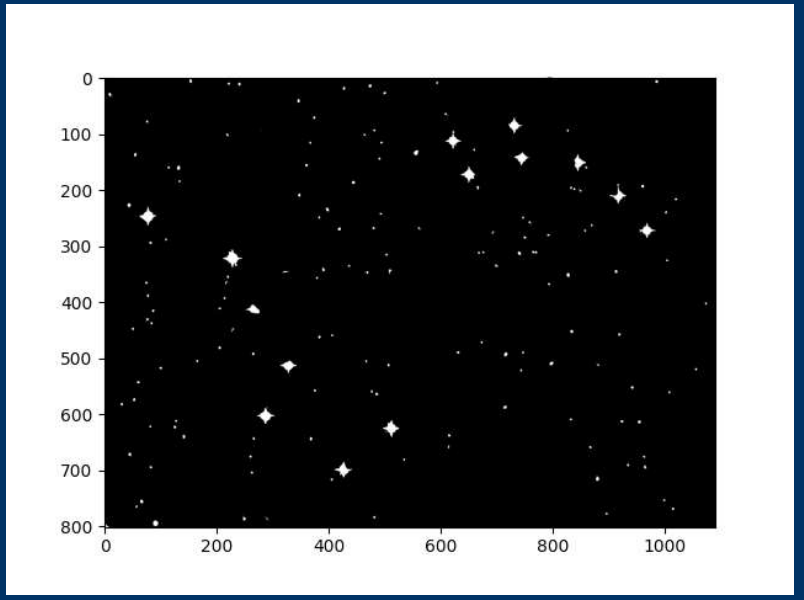
III) Expérience sur des constellations réelles
- 1) Protocole expérimental
- 2) Résultat

IV) Validation du cahier des charges

I) 1) Protocole de détection de constellation : acquisition



Filtrage



Relevé des coordonnées :

```
points  = [(182, 132), (122, 158), (134, 184), (126, 145), (570, 996), (572, 903),
(530, 989), (599, 908), (128, 115), (194, 106), (148, 168), (173, 110), (140, 150),
(512, 998), (570, 935), (577, 976), (573, 911), (592, 938), (529, 162), (508, 117),
(542, 194), (520, 120), (511, 964), (557, 958), (540, 969), (523, 963), (551, 915),
(594, 927), (538, 946), (502, 917), (544, 992), (513, 930), (510, 957), (587, 903),
(550, 985), (579, 914), (527, 952), (543, 933), (572, 943), (539, 945), (594, 183),
(507, 106), (508, 137), (508, 130), (101, 940), (164, 945), (197, 958), (198, 935),
```

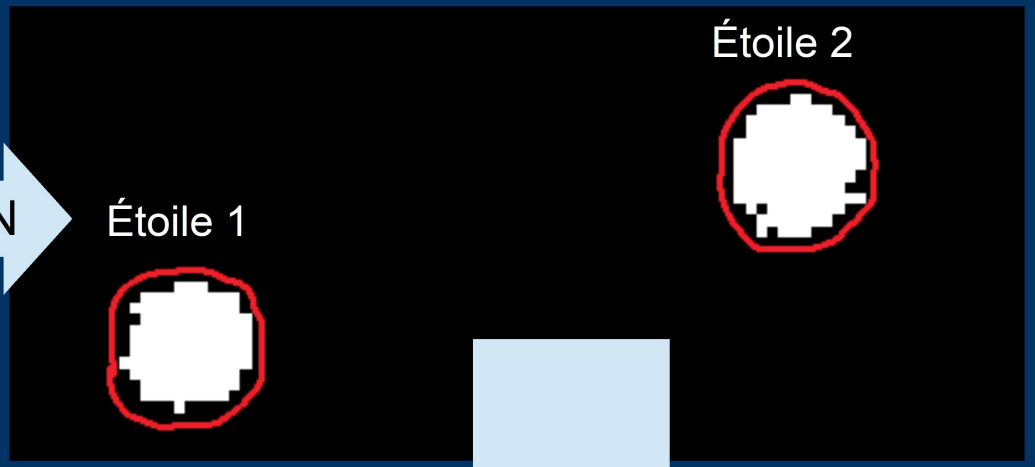
https://www.numerama.com/sciences/2148155-tout-savoir-sur-la-constellation-de-la-petite-ourse.html

I) 2) Protocole de détection : groupage

https://img4.linternaute.com/YHGOfI2ENMrI4iwdgJZ3pAA8kZA.jpg

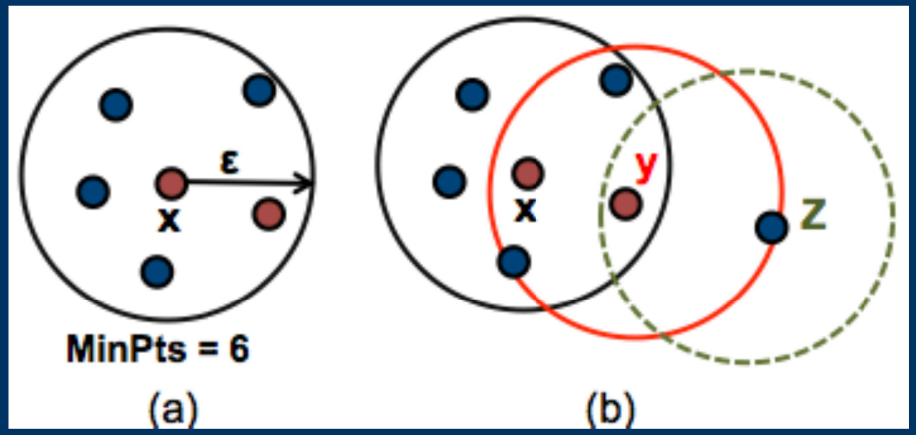


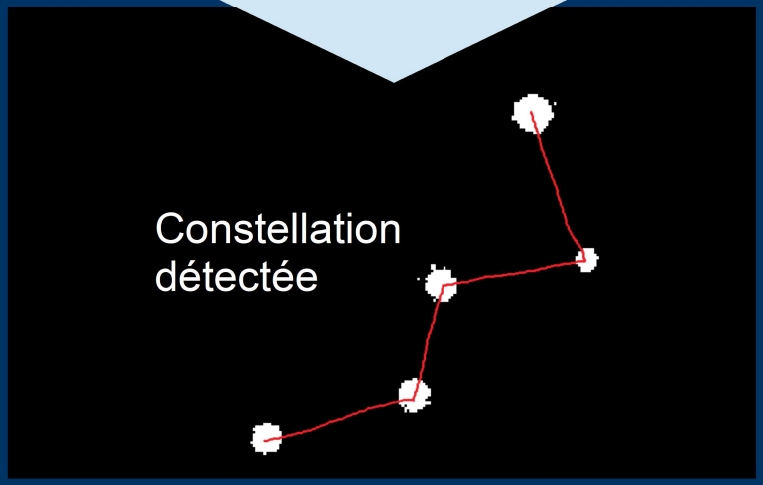
DBSCAN



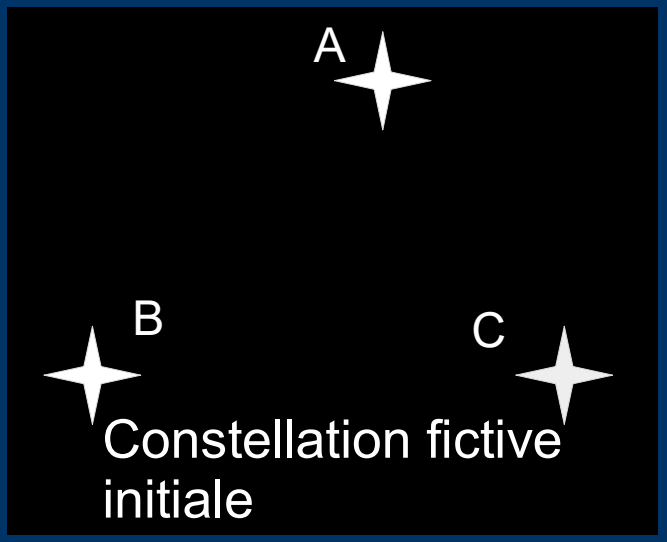
DBSCAN

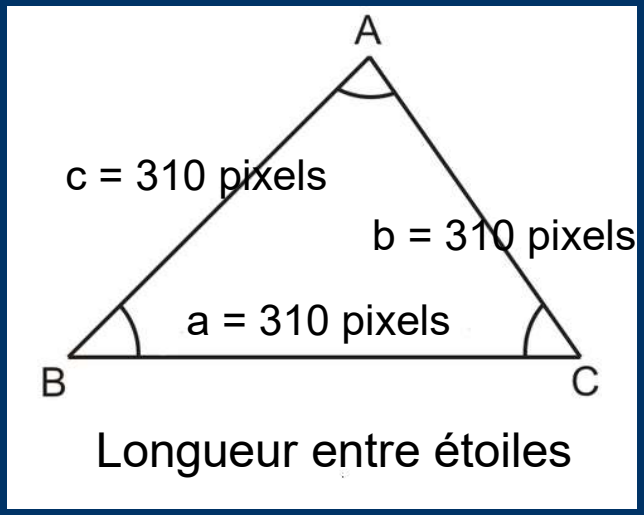
DBSCAN(MinPts, ε) :

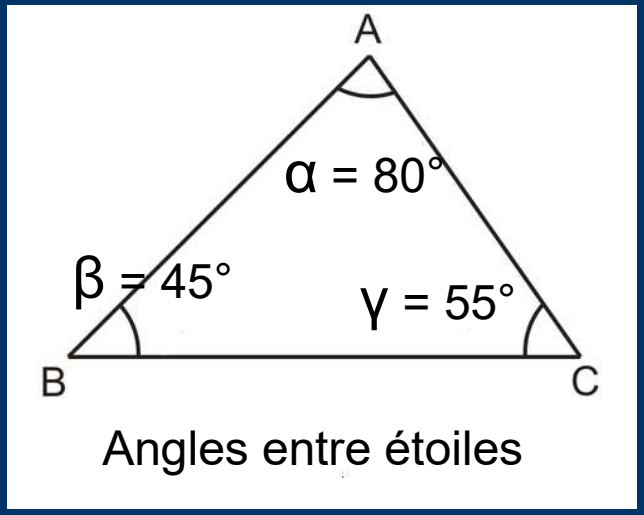




I) 3) Protocole de détection : caractérisation







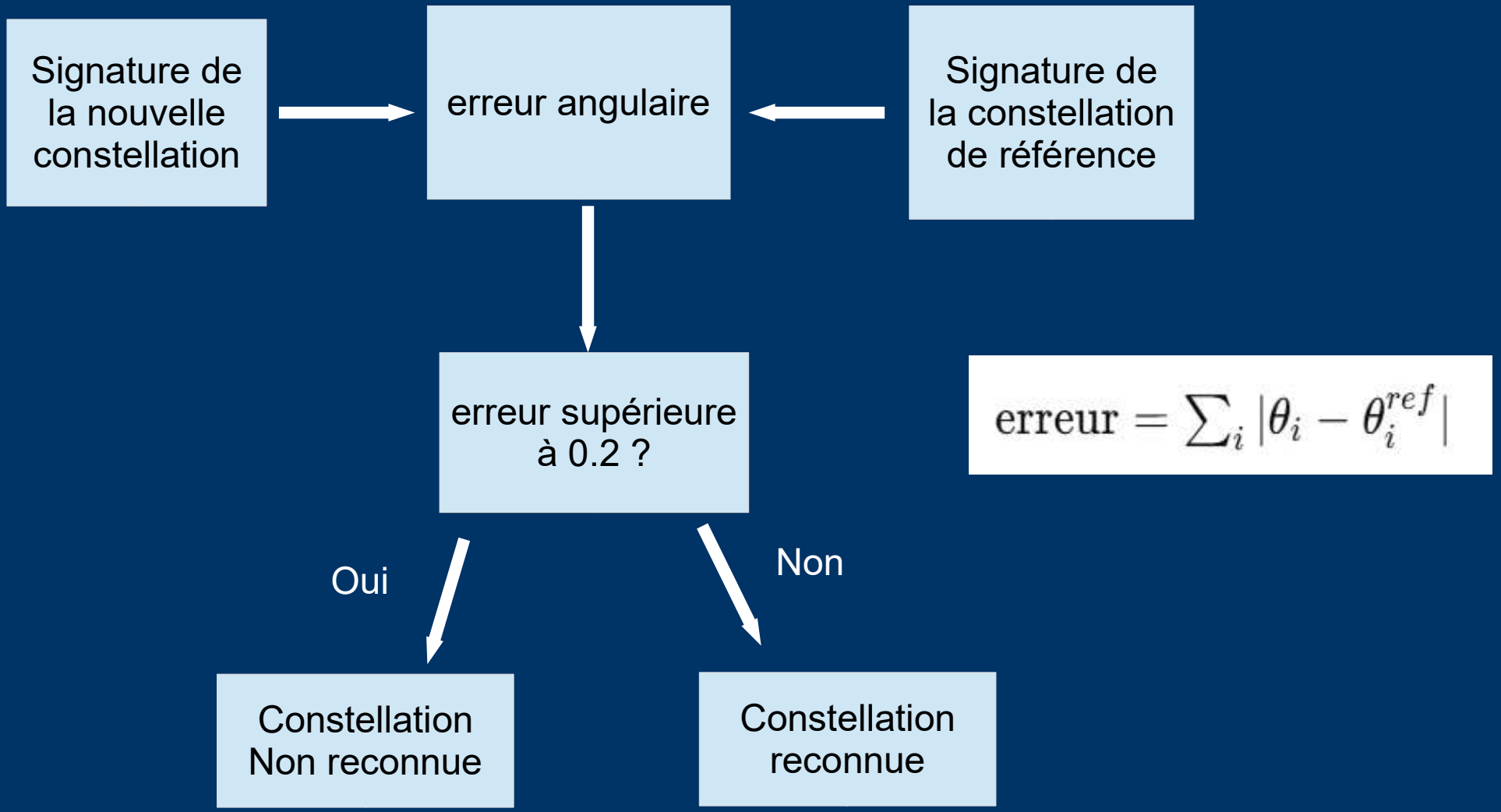
Loi des cosinus : $a^2 = b^2+c^2-2\cos(\alpha)$

$b^2 = a^2+c^2-2\cos(\beta)$

$c^2 = a^2+b^2-2\cos(\gamma)$

Signature géométrique : $\Theta = (\theta 1, \theta 2, \theta 3, \ldots)$

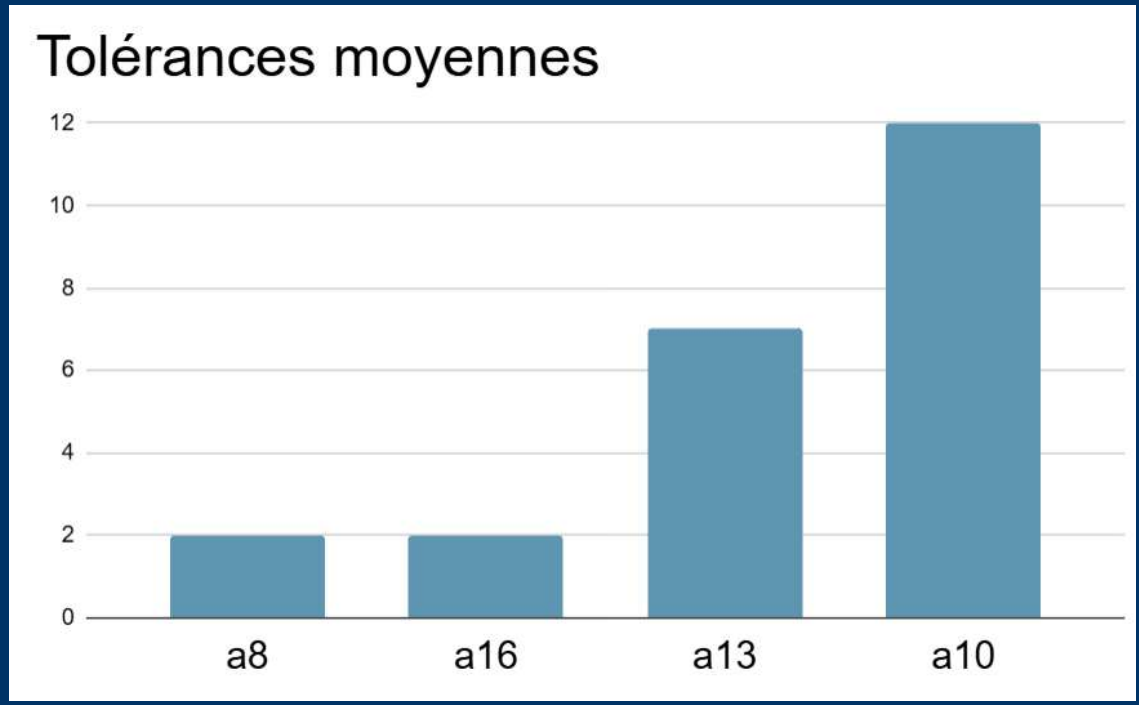
I) 4) Protocole de détection : reconnaissance



$$\text{erreur} = \sum_i |\theta_i - \theta_i^{ref}|$$

II) 1) Simulation : insensibilité aux perturbations

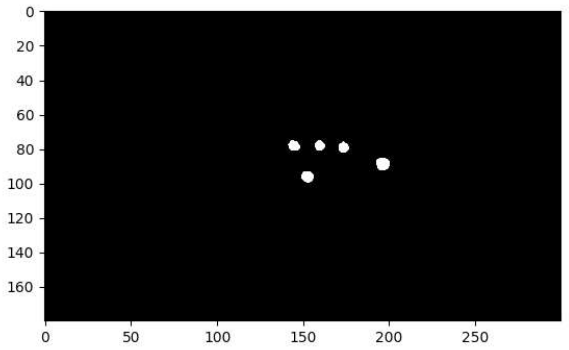
- Tolérance : capacité du programme à reconnaître des images déformées
- Résolution : 280 par 180



Tolérance	x1,y1	x2,y2	x3,y3	x4,y4	x5,y5	Moyenne
a8	(7,1)	(0,3)	(3,3)	(0,1)	(2,0)	2.0

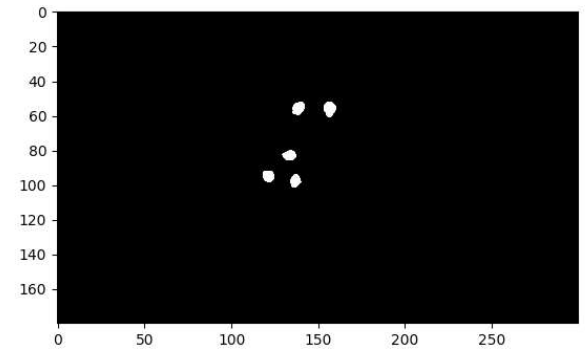
II) 1) Simulation : insensibilité aux perturbations

Constellation a8



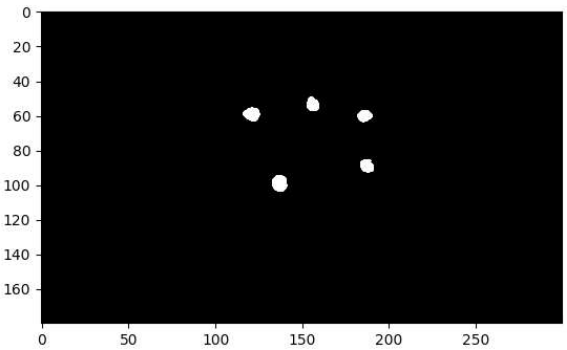
Tolérance : 2.0

Constellation a16



Tolérance : 1.9

Constellation a13

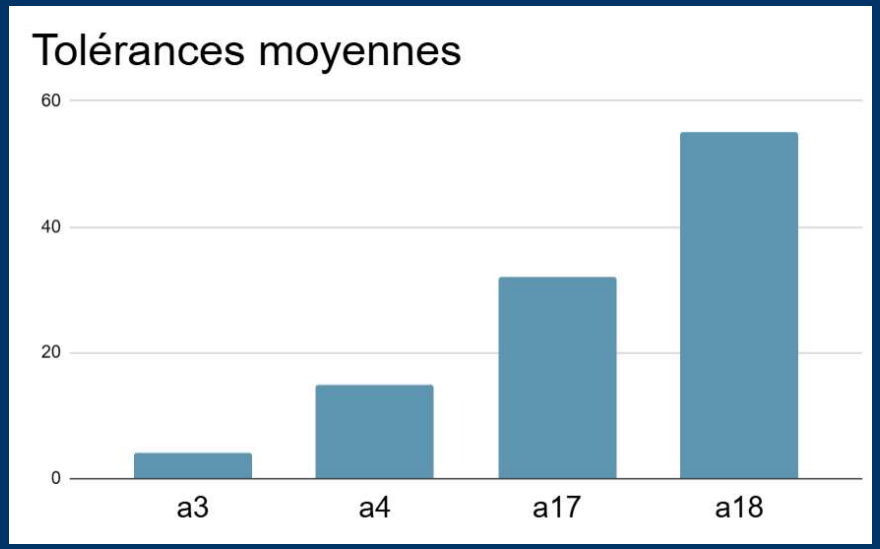


Tolérance : 7.1

II) 1) Simulation : insensibilité aux perturbations

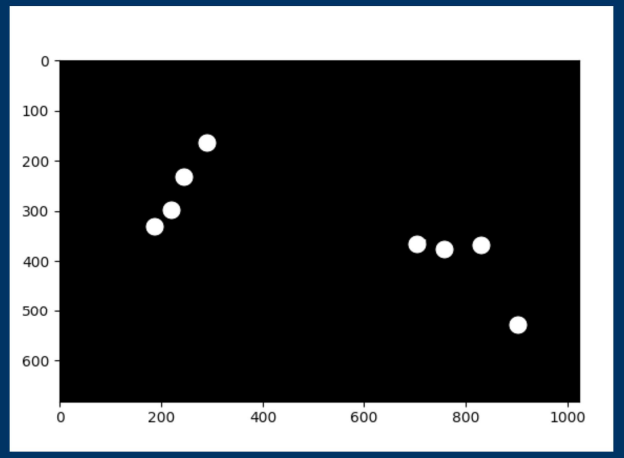
- **Plusieurs constellations par image**
- Résolution des images : 1024 par 648

Résultats :



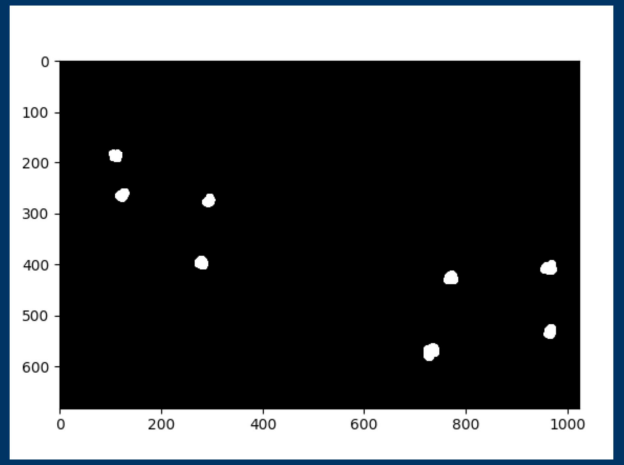
Constellations a3 et a4

Pire cas du programme



Constellations a17 et a18

Meilleur cas du programme



II) 2) Simulation : taux de détection : protocole et résultats

	Taux de détection de constellations
Simulation 1	0.8157
Simulation 2	0.8199
Simulation 3	0.8158
Simulation 4	0.8173
Simulation 5	0.8189
Moyenne	0.8175

- **N = 10000 essais**
- Tendance des constellations à se superposer : limite l'efficacité

III) 1) Expérience : protocole

- Base de donnée de constellations réelles
- Taux de détection

Photo de la Grande et de la Petite Ourse



https://www.numerama.com/sciences/2148155-tout-savoir-sur-la-constellation-de-la-petite-ourse.html

Photo de 7 constellations (retouchée)

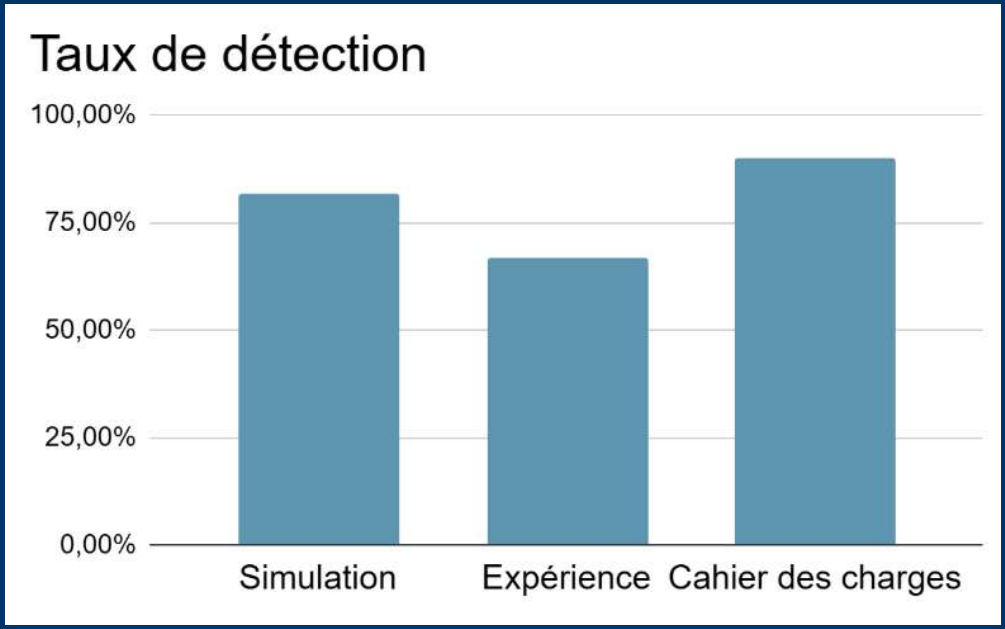


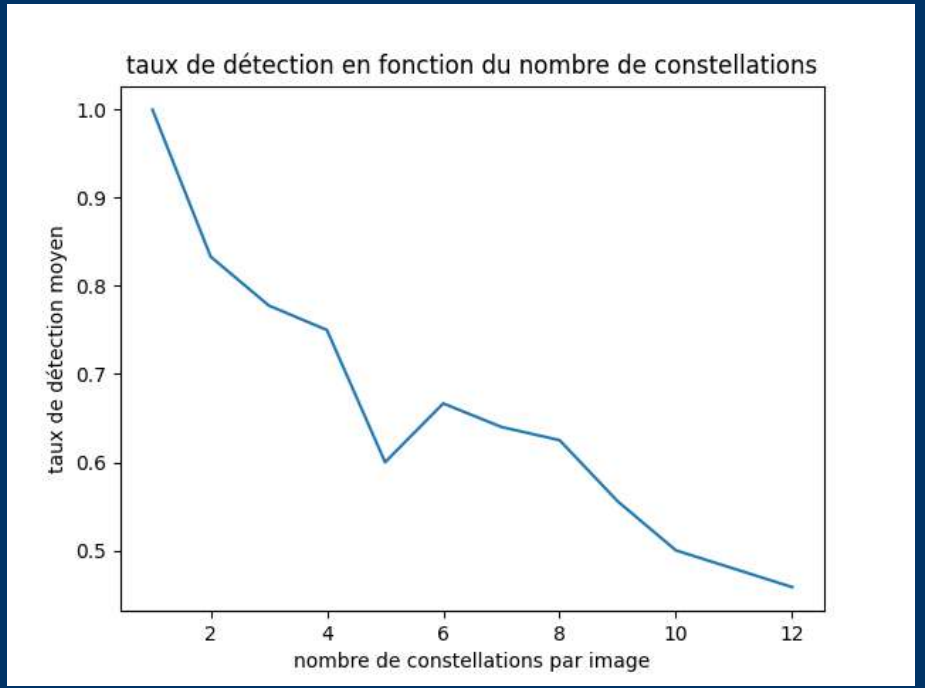
https://www.istockphoto.com/fr/search/2/image-filmphrase=constellation&tracked_gsrp_landing

III) 2) Expérience : résultats

Taux de détection expérimental : 66.5%

- Nombre de tests : 21
- 100% de détection sur des images contenant 1 constellation





IV) Validation du cahier des charges

Performance	Cahier des charges	Simulation	Expérience
Insensibilité aux perturbations	Constellations reconnaissables lorsqu'elles diffèrent de 1.0% de l'image de référence	Entre 10 et 50 pixels	Non mesurable (faible)
Taux de détection	T > 90%	82.0% < 90%	66.5% < 90%
Temps d'exécution	Tr < 1.0s	0.005 < Tr < 0.010	0.005 < Tr < 0.010
Conclusion		**Reconnaissance fonctionnelle mais insuffisante**	

IV) Conclusion : Limites et Améliorations possibles

Limites	Améliorations possibles
- chevauchement des constellations - étoiles manquantes - pollution lumineuse - base de données limitée	- pondération par luminosité - utilisation des distances relatives - apprentissage automatique

Fin

Merci pour votre attention

Annexe : code (1/14) : Importation et filtrage

```
583  import os
584  from numpy import *
585  from matplotlib.pyplot import *
586  import matplotlib.image as mpimg
587  from sklearn.cluster import DBSCAN
588  from random import *
589  from itertools import permutations
590  import math
591
592
593  os.chdir("C://Users//achau//Documents//TIPE astro//constellations brutes")
594  photo = mpimg.imread("img_cassiopée.png")
595  imshow(photo)
596  show()
597
598
599  # filtrage des images: on efface l'arrière-plan et on récupère les coordonnées des points lumineux
600
601
602  def affiche(Photo): # Prend en argument une image Python, (une matrice)
603      m, n = Photo.shape[:2]
604      coord1 = [] # liste de listes à 2 éléments
605      for i in range(m):
606          for j in range(n):
607              if 0.2 <= Photo[i,j,0] <= 1.0 and 0.2 <= Photo[i,j,1] <= 1.0 and 0.2 <= Photo[i,j,2] <= 1.0: #
608                  coord1.append([i, j])
609              else: # le pixel doit être remplacé.
610                  Photo[i,j,0] = 0 # on remplace par un fond noir l'arrière-plan
611                  Photo[i,j,1] = 0
612                  Photo[i,j,2] = 0
613      return(Photo, coord1) # renvoie la même image sans l'arrière-plan + les coordonnées des étoiles
614
615
616  photo, coord1 = affiche(photo)
617  imshow(photo)
618  show()
```

Annexe : code (2/14) : Groupage des points brillants en étoiles

```
621 def separer_etoiles(L, eps = 80, min_samples = 60):
622     """
623     L : liste de coordonnées [(x1, y1), (x2, y2), ...]
624     eps : distance maximale entre deux points pour être considérés comme voisins
625     min_samples : nombre minimum de points pour former une étoile (cluster)
626     """
627     if not L:
628         return []
629     points = array(L)
630     clustering = DBSCAN(eps = eps, min_samples = min_samples).fit(points)
631     etoiles = {}
632     for label, point in zip(clustering.labels_, L):
633         if label == -1:
634             continue  # Bruit, point non assigné
635         etoiles.setdefault(label, []).append(point)
636     return list(etoiles.values())
637
638
639 L_grosses_étoiles = separer_etoiles(coord1)
```

Annexe : code (3/14) : Coordonnées des centres de chaque étoiles

```
658
659
660     def coord_moyenne(L): # L une liste de liste de coordonnées
661         n = len(L)
662         xtot, ytot = 0, 0
663         Lmoy = []
664         for i in range(n): # pour chaque étoile...
665             m = len(L[i])
666             for j in range(m):
667                 xtot += L[i][j][0]
668                 ytot += L[i][j][1]
669             xtot /= m
670             ytot /= m
671             Lmoy.append([xtot, ytot])
672         return(Lmoy)
673
674
675     L_moy = coord_moyenne(L_grosses_étoiles)
676     nb_étoile = len(L_moy)
677     print(nb_étoile)
678
```

Annexe : code (4/14) : Séparation des étoiles en constellations

```
679
680     coord_vrac = L_moy
681     X = array(coord_vrac)
682     ident_N_const = DBSCAN(eps=80, min_samples=5)
683     labels = list(ident_N_const.fit_predict(X))
684     c = {}
685     for i in range(len(labels)):
686         c[str(labels[i])] = []
687     for i in range(len(labels)): # liste que associe à chaque label les
688                                  # coordonnées correspondantes des étoiles
689         c[str(labels[i])].append(coord_vrac[i])
690     c = list(c.values()) #liste sans les labels, qui ne sont pas voulus
691     print(c)
692     print(labels)
693
```

Annexe : code (5/14) : Calcul des angles

```
694
695     def angle_entre(p1, p2, p3):
696         # Vecteurs BA et BC
697         ba = (p1[0] - p2[0], p1[1] - p2[1])
698         bc = (p3[0] - p2[0], p3[1] - p2[1])
699         # Produit scalaire
700         dot = ba[0]*bc[0] + ba[1]*bc[1]
701         # Normes des vecteurs
702         norm_ba = math.hypot(*ba)
703         norm_bc = math.hypot(*bc)
704         if norm_ba == 0 or norm_bc == 0:
705             return None  # Vecteur nul, angle indéfini
706         # Calcul de l'angle en radians
707         cos_theta = dot / (norm_ba * norm_bc)
708         angle = math.acos(cos_theta)
709         # Convertir en degrés
710         return math.degrees(angle)
711
```

Annexe : code (6/14) : Calcul des angles

```
430
431 def tous_les_angles(L):
432     T = []
433     # Générer toutes les permutations de 3 points distincts
434     for i, j, k in permutations(range(len(L)), 3):
435         p1, p2, p3 = L[i], L[j], L[k]
436         angle = angle_entre(p1, p2, p3)
437         if angle is not None:
438             # Stocker sous forme : (indices, angle)
439             T.append(((i, j, k), angle))
440     return T # renvoie la liste des angles + triplets d'étoiles.
441
442
443 angl = []
444 for Ci in c:
445     angl.append(tous_les_angles(Ci))
446
```

Annexe : code (7/14) : Élimination des doublons

```
447
448     def doublons_angl(L):
449         L1, L2 = [], []
450         for i in range(len(L)):
451             if L[i][1] in L1:
452                 continue
453             else:
454                 L1.append(L[i][1])
455                 L2.append(L[i])
456         return(L2) # liste sans doublons
457
458
459     for i in range(len(angl)):
460         angl[i] = doublons_angl(angl[i])
461
```

Annexe : code (8/14) : Comparaison par nombre d'étoiles

```
475
476 def comparer(angles, nb_ét, base, nb_étoiles):
477     """
478     "angles" la liste des angles de l'étoile observée,
479     "nb_ét" le nombre de points de l'étoile observée,
480     "base" et "nb_étoiles" deux dictionnaires
481     """
482     # on compare avec les nombres de points
483     dic = {} # dictionnaire des éléments restants
484     for clé, valeur in nb_étoiles.items():
485         if valeur == nb_ét: # constellations de même nombre d'étoile
486             dic[clé] = base[clé] # constellation sous forme d'angles
487     a = "étoiles possibles: "
488     for i in dic:
489         a = a + i + ', '
490     L2 = [] # liste des angles comparants
491     for i in range(len(angles)):
492         L2.append(angles[i][1])
493     L2.sort() # tri des angles par valeurs croissantes
494
```

Annexe : code (9/14) : Comparaison par les angles

```
496     echec, succes = True, []
497     for clé, valeur in dic.items(): # parcours de la base de donnée REDUITE
498         L1 = [] # liste des angles comparés des constellations connues de la base
499         for i in range(len(valeur)): # parcours des angles
500             L1.append(valeur[i][1])
501         L1.sort()
502         # on compare 2 à 2 les angles, en espérant qu'ils soient très similaires
503         ecarts_relatifs = []
504         for i in range(len(angles)):
505             if L2[i] == 0: # cas de l'angle nul (division par 0)
506                 ecarts_relatifs.append(0)
507             else:
508                 ecarts_relatifs.append(abs(L2[i] - L1[i]) / L2[i])
509         correspondance = True
510         for i in range(len(angles)):
511             if ecarts_relatifs[i] >= 0.20: # constellation reconnue si son écart est minimal
512                 correspondance = False
513         if correspondance == True:
514             succes.append(clé) # liste des constellations reconnues
515             echec = False
516     if echec == True:
517         print("pas de const reconnu")
518     print("probablement: "+str(succes))
519     return(succes)
520
```

Annexe : code (10/14) : Comparaison de chaque constellation de l'image

```
521
522 # répétition du processus pour chaque constellation de l'image
523 L = []
524 for i in range(len(angl)):
525     L = L + comparer(angl[i], len(c[i]), base, nb_étoiles)
526 print("")
527 print('total:', L)
528
```

Annexe : code (11/14) : Génération de constellations aléatoires

```
269 def ordre_alea(n): # pour n images
270     L = []
271     for i in range(n): # chaque image
272         L.append([])
273         for j in range(4):
274             """
275             les constellations doivent être réparties aux 4 coins de l'image
276             """
277             a = randint(1,4)
278             while a in L:
279                 a = randint(1,4)
280             L[i].append(a)
281         for j in range(4):
282             if L[i][j] == 1:
283                 L[i][j] = [100, 200, 100, 200]
284             elif L[i][j] == 2:
285                 L[i][j] = [100, 200, 900, 1000]
286             elif L[i][j] == 3:
287                 L[i][j] = [500, 600, 100, 200]
288             elif L[i][j] == 4:
289                 L[i][j] = [500, 600, 900, 1000]
290     return(L)
291
```

Annexe : code (12/14) : Génération de constellations aléatoires

```
320
321 def pattern_alea(nb):
322     """
323     'nb' nombre de constellation aléatoires à générer
324     """
325     ordre = ordre_alea(20)
326     L_img, validation = [], []
327     cpt = 0
328     for k in range(nb): # chaque image
329         nb_const_par_img = randint(1,4)
330 _       img, T_img = [], []
331         for i in range(nb_const_par_img): # chaque constellation
332             cpt += 1
333             nb_ét_par_const = randint(4,5)
334 _           if randint(1,5) == 6:
335 _               vraie = False # on invente une nouvelle constellation
336                 const = []
337                 for j in range(nb_ét_par_const): # chaque étoile de la constellation
338                     ét = (randint(ordre[k][i][0], ordre[k][i][1]), randint(ordre[k][i][2], ordre[k][i][3]))
339 _                   const.append(ét)
340                 nom = 'inconnu'
341             else:
342                 vraie = True # on choisi une constellation aléatoire parmi les 45 de la base de donnée
343                 b = randint(1,29)
344 _               const = points['a'+str(b)]
345                 nom = 'a'+str(b)
346             img.append(const)
347             T_const = [vraie, nb_ét_par_const, nom]
348             """[0: const présente ou non dans la base de donnée, 1: nombre d'étoiles, 2: nom]"""
349             T_img.append(T_const)
350         L_img.append(img)
351         validation.append(T_img) # liste de validation des résultats
352     return(L_img, validation) # renvoie une liste de constellations aléatoires et une liste de validation
353
```

Annexe : code (13/14) : Vérification des résultats simulés

```
530
531 def verif(n):
532     "n: le nombre d'essais"
533     cpt_succes = 0
534     for k in range(n): # pour chaque image aléatoire
535         coord0 = pattern_alea(1) # coordonnées d'une photo
536         coord_vrac = []
537 
538         # exécution du processus entier d'analyse
539         for i in range(len(coord0[0][0])):
540             coord_vrac += coord0[0][0][i]
541         X = array(coord_vrac)
542         ident_N_const = DBSCAN(eps=80, min_samples=60)
543         labels = list(ident_N_const.fit_predict(X)) # groupage des constellations
544         c = {}
545         for i in range(len(labels)):
546             c[str(labels[i])] = []
547         for i in range(len(labels)):
548             c[str(labels[i])].append(coord_vrac[i])
549         c = list(c.values()) # liste des coordonnées par constellation sans les labels
550         angl = []
551         for Ci in c: # angles bruts entre chaque étoiles de chauqe constellations
552             angl.append(tous_les_angles(Ci))
553         for i in range(len(angl)): # élimination des doublons d'angles
554             angl[i] = doublons_angl(angl[i])
555         L = [] # constellations reconnues (résultat)
556         for i in range(len(angl)): # comparaison
557             L = L + comparer(angl[i], len(c[i]), base, nb étoiles)
```

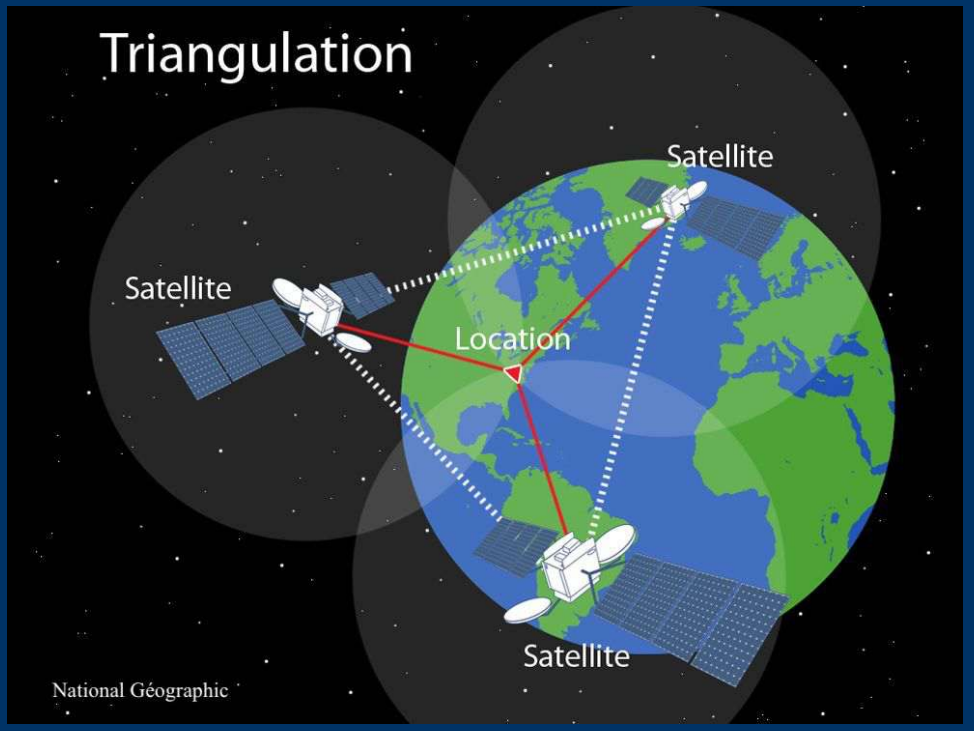
Annexe : code (14/14) : Vérification des résultats simulés

```
558
559         validation = coord0[1] # jeux de donnée permettant de valider le résultat
560         const_valid = [] # liste des constellations modeles
561         for i in range(len(validation[0])):
562             const_valid.append(validation[0][i][2])
563         if L == const_valid: # reconnaissance de chaque constellation de l'image
564             cpt_succes += 1
565         elif L == []: # aucune constellations détectées
566             cpt_succes += 0
567         elif set(L).issubset(set(const_valid)) == True: # s'il manque des constellations
568             cpt_succes += len(L) / len(const_valid) # proportion de const reconnues sur l'image
569         elif set(const_valid).issubset(set(L)) == True: # constellations reconnues non présentes
570             cpt_succes += 1
571         print('photo:', k+1, ' modèle:', const_valid, ' réalité:', L)
572     return(cpt_succes/n) # taux de réussite
573
574 print('taux de réussite:', verif(10000))
575
```

Annexe : Fonctionnement du GPS par triangulation

Mesure de distance par utilisation des ondes :
D = C * ΔT / 2

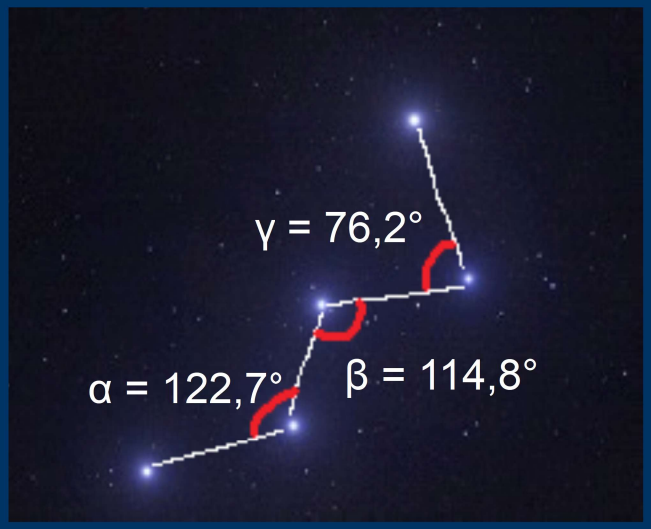
Avec :
- C : vitesse de l'onde radio
(3.0 * 10**8 m/s)
- ΔT : temps d'aller-retour du signal
- D : distance point-satellite



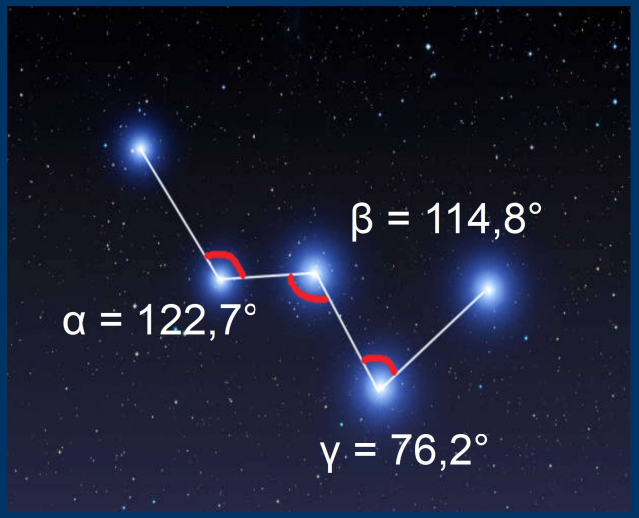
Méthode :
- 3 distances satellite / point à localiser
- position des 3 satellites connues
- Le point à localiser se trouve à l'intersection des 3 sphères

Annexe : invariance des signatures par rotation

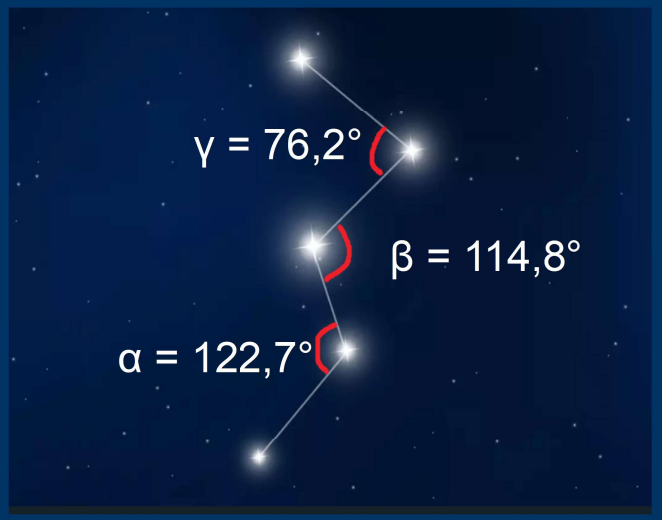
Images de la constellation de Cassiopée suivant différentes orientations :



https://img4.linternaute.com/YHGOfI2ENMrI4iwdgJZ3pAA8kZA.jpg



https://media.istockphoto.com/id/1186728772/fr/photo/constellation-de-cassiopeia-sur-le-ciel-%C3%A9toile%C3%A9.jpg

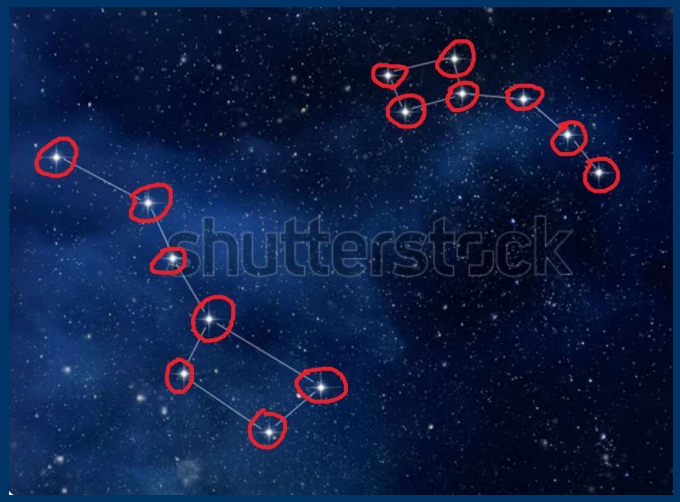


https://img4.linternaute.com/CmdpzIcZ2IttoGeId3u6aPOCLi8=/1080x/smart/b4605556ca9d49bb8555c8b62603aafe/ccmcms-linternaute/35851863.jpg

Annexe : Sensibilité des paramètres de DBSCAN

Utilisés durant les expériences : MinPts = 60 et Epsilon = 80

Epsilon = 10



Mauvais clustering

Epsilon = 600



Mauvais clustering

Epsilon = 80



Clustering approprié

Annexe : complexité de l'algorithme

P : nombre de pixels par image
Q : nombre de points brillants avec P>>Q et Q>>N
N : nombre d'étoiles

Fonction	Complexité
Fonction de filtrage	En $O(P^2)$ (quadratique)
Fonction de séparation des points brillants en étoiles	En O(Qlog(Q)) (quasi-linéaire)
Calcul des centres de chaque étoiles	En $O(N^2)$
Séparation des constellations	En $O(N^2)$
Calcul des angles	En $O(N^2)$
Comparaison	En $O(N^2)$
Vérification des résultats	En $O(N^2)$
Génération de constellations fictives aléatoires	En $O(N^2)$
Algorithme total	En $O(P^2)$

Annexe : alternative à la fonction de clustering DBSCAN : algorithme des K-moyennes

Avantage de DBSCAN : ne nécessite pas de connaître le nombre de cluster à l'avance

```
k = 3 # à choisir manuellement
N = len(étoiles) # nombre d'étoiles
p = N//k
# initialisation des clusters
centroides = {}
Etoiles_clusters = {}
# Etoiles_clusters est une liste de deux éléments : les données à traiter et les données traitées
for i in range(k-1): # initialisation : construction arbitraire des clusters
    Etoiles_clusters[i] = [étoiles[i*p:(i+1)*p,:],[]] # La liste des deux tableaux de données classées à n et n+1
    centroides[i] = étoiles[i*p,:] # la première valeur par exemple

Etoiles_clusters[k-1] = [étoiles[(i+1)*p:N,:],[]] # Le dernier cluster contiend les données restantes
centroides[k-1] = étoiles[(k-1)*p,:] # Initialisation du dernier cluster


# fonction de calcul des centroides et des clusters
def CalculCentroides(Etoiles_clusters, centroides):
    for CleVal, DataVal in Etoiles_clusters.items():
        centroides[CleVal] = np.mean(DataVal[1],0) # retourne les centroïdes des nouveaux clusters
        Etoiles_clusters[CleVal][0], Etoiles_clusters[CleVal][1] = Etoiles_clusters[CleVal][1],[] # mise à jour des clusters
    return Etoiles_clusters, centroides


# fonction de clustering
def Clusterisation(Etoiles_clusters, centroides,T):
    T = True # variable détectant la répétition des clusters
    for CleVal, DataVal in Etoiles_clusters.items(): # pour chaque cluster
        for donnees in DataVal[0]:
            classe = [float(np.inf),0,[]] # initialisation pour la recherche de la meilleure classe
            for CleCentroide, Centroide in centroides.items(): # calcul de la donnée la plus proche
                d = np.dot(donnees-Centroide, donnees-Centroide)
                if d <= classe[0]:
                    classe = [d, CleCentroide, donnees]
            if CleVal != classe[1]:
                T = False # DataVal a changé de cluster
            Etoiles_clusters[classe[1]][1].append(classe[2]) # Mise à jour de la classe : ajout de la donnée à la nouvelle classe
    Etoiles_clusters, centroides = CalculCentroides(Etoiles_clusters, centroides) # recalcul des centroides
    return Etoiles_clusters, centroides, T


# fonction principale d'application du k-moyenne
def kmean(Etoiles_clusters, centroides):
    T = False
    while not(T): # répétition du processus jusqu'à ce que les clusters n'évoluent plus
        Etoiles_clusters, centroides, T = Clusterisation(Etoiles_clusters, centroides,T)
    return Etoiles_clusters, centroides


Etoiles_clusters, centroides = kmean(Etoiles_clusters, centroides) # appel de l'algorithme
```